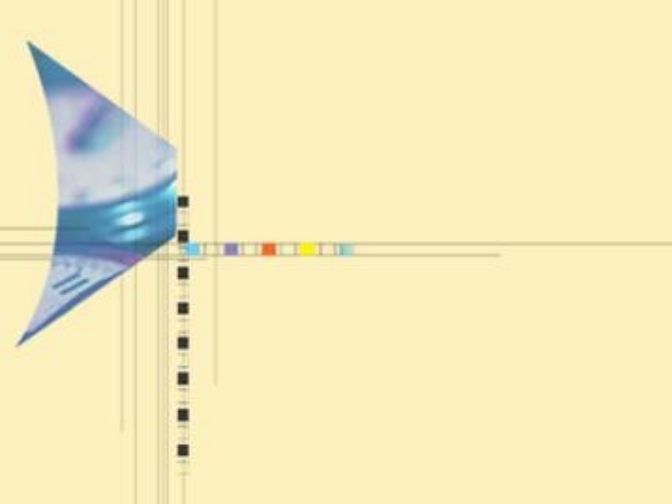




算法设计与分析

主讲：刘娟，武汉大学人工智能学院

2024-2025 第二学期，1-16周

周一(3-4，三区1-502)，周三（6-7，三区1-325）

第三讲

分 治



主要内容



3.1 引言

3.2 分治算法的简单例

3.3 分治策略的思想

3.4 分治策略的经典应用

3.5 分治算法的改进



主要内容



3.4. 分治策略的经典应用（选讲）

3.4.1 寻找第k小元素

3.4.2 快速排序

3.4.3 多数元素

3.4.4 芯片测试问题

3.4.5 循环赛日程表

3.4.6 求向量卷积

3.4.7 棋盘覆盖问题

3.4.8 最大子序列和问题



3.4.1 选择问题：寻找第k小元素

问题描述：在一个包含n个元素的集合中寻找第k个最小元素。
(中值是第 $\lceil n/2 \rceil$ 个最小元素， $k=1$ 最小元素， $k=n$ 最大元素)

- 直接方法：

- 对所有的元素非降排序，取出第k个位置元素。这个方法需要 $\Omega(n \log n)$ 时间，因为任何基于比较的排序过程在最坏的情况下必须至少要花费这么多时间

能否在线性时间内找到更快的算法？

SELECT算法

算法思想的动机:

- 假设递归算法中，在每个递归调用的划分步骤后，我们丢弃元素的一个固定部分并且对剩余的元素递归。则问题的规模以几何级数递减。也就是，在每个调用过程中，问题的规模以一个常因子被减小。
 - 例如：某个算法丢弃1/3并对剩余的2/3部分递归，那么在第二次调用中，元素的数目变为 $2n/3$ 个，第三次调用中为 $4n/9$ ，第四次调用中变为 $8n/27$ ，等等。
- 现在假定在每次调用中，算法对每个元素耗费的时间不超过一个常数，则耗费在处理所有元素上的全部时间产生一个几何级数，如：

$$cn + (2/3)cn + (2/3)^2 cn + \dots + (2/3)^j cn + \dots$$

这里。 c 是某个被适当选择的常数，这个总数小于

$$\sum_{j=0}^{\infty} cn(2/3)^j = 3cn = \Theta(n)$$

SELECT算法

算法基本思想

- 首先，如果元素个数小于阈值44，则直接方法求第k小元素；否则，进入下一步。
- 将n个元素划分成 $\lfloor n/5 \rfloor$ 组，每组由5个元素组成。如果n不是5的倍数，则排出剩余元素，则应该不影响算法的执行。
- 将每组进行排序，并取出中项元素，即第3个元素。
- 通过递归调用的方式求出这些中项元素中的中项元素，记为mm。
- 根据同mm的大小关系，将数组A中元素划分成三个子数组：A1, A2, A3, 其中分别包含小于，等于和大于mm的元素。
- 确定A中第k小元素出现在三个数组的哪一个，并根据测试结果，要么返回第k小元素，要么在A1或A3上进行递归。



举例

假设含有25个元素的数组 $A[1..25] =$

$\{8, 33, 17, 51, 57, 49, 35, 11, 25, 37, 14, 3, 2, 13, 32, 12, 6, 29, 32, 54, 5, 16, 22, 23, 7\}$

要求找出数组A中第13小的元素。

为了例子演示方便，临时将算法中阈值从44改为6。



8	33	17	51	57	49	35	11	25	37	14	3	2	13	52	12	6	29	32	54	5	16	22	23	7
---	----	----	----	----	----	----	----	----	----	----	---	---	----	----	----	---	----	----	----	---	----	----	----	---

partition

8	33	17	51	57	49	35	11	25	37	14	3	2	13	52	12	6	29	32	54	5	16	22	23	7
---	----	----	----	----	----	----	----	----	----	----	---	---	----	----	----	---	----	----	----	---	----	----	----	---

sort

8	17	33	51	57	11	25	35	37	49	2	3	13	14	52	6	12	29	32	54	5	7	16	22	23
---	----	----	----	----	----	----	----	----	----	---	---	----	----	----	---	----	----	----	----	---	---	----	----	----

median

13	16	29	33	35
----	----	----	----	----

partition

8	17	11	25	14	3	2	13	12	6	5	16	22	23	7	29	33	51	57	49	35	37	52	32	54
---	----	----	----	----	---	---	----	----	---	---	----	----	----	---	----	----	----	----	----	----	----	----	----	----

discard

A[1...15]

8	17	11	25	14	3	2	13	12	6	5	16	22	23	7
---	----	----	----	----	---	---	----	----	---	---	----	----	----	---

partition

8	17	11	25	14
---	----	----	----	----

3	2	13	12	6
---	---	----	----	---

5	16	22	23	7
---	----	----	----	---

sort

8	11	14	17	25
---	----	----	----	----

2	3	6	12	13
---	---	---	----	----

5	7	16	22	23
---	---	----	----	----

median

6	14	16
---	----	----

partition

8	11	3	2	13	12	6	5	7
---	----	---	---	----	----	---	---	---

14

17	25	16	22	23
----	----	----	----	----

discard





A[11...15]

17	25	16	22	23
----	----	----	----	----

sort

16	17	22	23	25
----	----	----	----	----

Succeed! The result is A[13]=22



算法3.4 SELECT

输入: n 个元素的数组 $A[1..n]$ 和整数 $k, 1 \leq k \leq n$ 。

输出: A 中的第 k 小元素。

1. $\text{select}(A, 1, n, k)$

过程 **select**($A, \text{low}, \text{high}, k$)

1. $p \leftarrow \text{high} - \text{low} + 1$

2. **if** $p < 44$ **then** 将 A 排序 **return** ($A[k]$)

3. Let $q = \lfloor p/5 \rfloor$. 将 A 分成 q 组, 每组5个元素。如果5不整除 p , 则排除剩余的元素

4. 将 q 组中的每一组单独排序, 找出中项。所有中项的集合为 M

5. $mm \leftarrow \text{select}(M, 1, q, \lceil q/2 \rceil)$ { mm 为中项集合的中项}

6. 将 $A[\text{low}..\text{high}]$ 分成三组:

$A1 = \{a | a < mm\}$ $A2 = \{a | a = mm\}$ $A3 = \{a | a > mm\}$

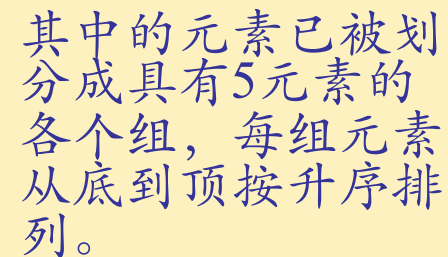
7. **case**

$|A1| \geq k$: **return** $\text{select}(A1, 1, |A1|, k)$

$|A1| + |A2| \geq k$: **return** mm

$|A1| + |A2| < k$: **return** $\text{select}(A3, 1, |A3|, k - |A1| - |A2|)$

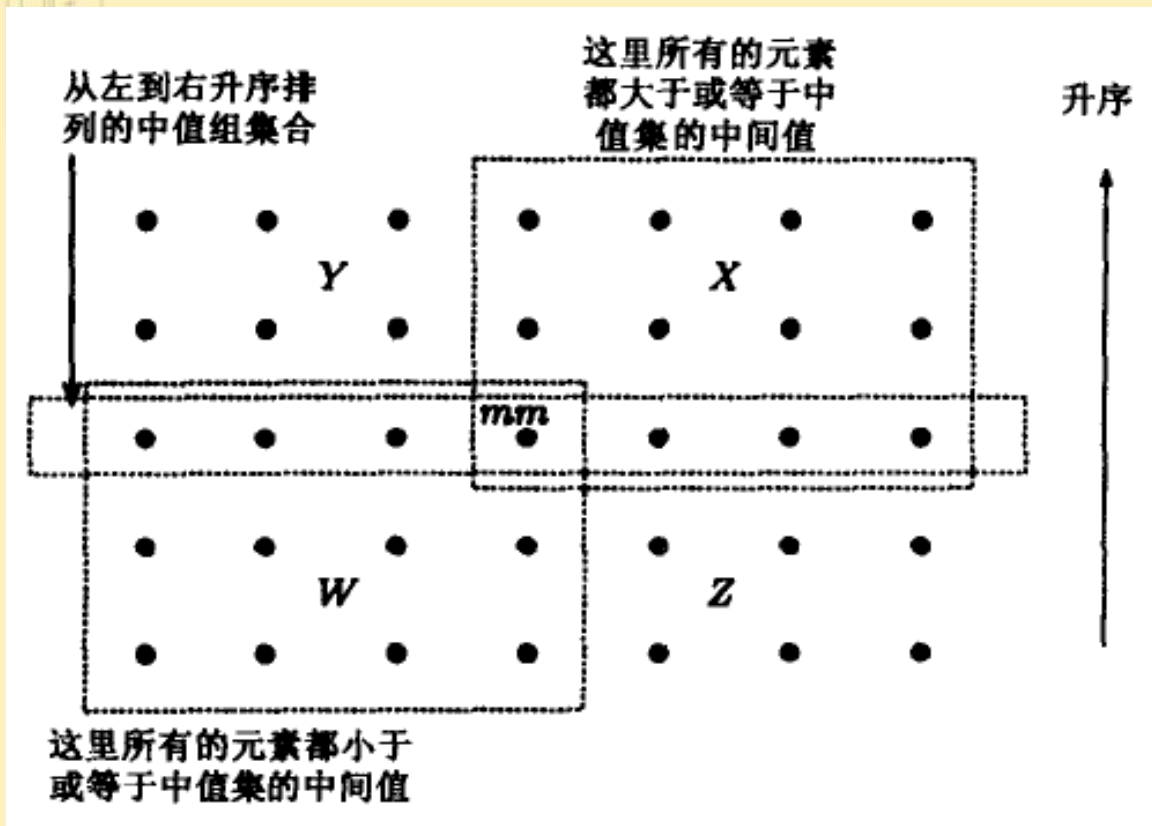
8. **end case**



而的且这此组以这样
的方式整齐排列:
它们的中项是从左
到右以升序排列的。



算法分析



所有围在矩形W中的元素是小于或等于mm的，所有围在矩形X中的元素是大于或等于mm的。

设 A'_1 表示小于或等于mm的元素集， A'_3 是大于或等于mm的元素集

注意：算法中， A_1 是严格小于mm的元素集， A_3 是严格大于mm的元素集

算法分析

- 由于 A'_1 至少与 W 同样大, 我们有

$$|A'_1| \geq 3\lceil \lfloor n/5 \rfloor / 2 \rceil \geq \frac{3}{2}\lfloor n/5 \rfloor$$

- 因此 $|A_3| \leq n - \frac{3}{2}\lfloor n/5 \rfloor \leq n - \frac{3}{2}\left(\frac{n-4}{5}\right) = n - 0.3n + 1.2 = 0.7n + 1.2$.

- 由参数的对称性得到

$$|A'_3| \geq \frac{3}{2}\lfloor n/5 \rfloor \text{ and } |A_1| \leq 0.7n + 1.2$$

- 这样就为在 A_1 和 A_3 的元素个数建立了上界:
 - 即小于 mm 的元素的个数和大于 mm 的元素个数不能够超过大约 $0.7n$ (n 的常分数倍)。

算法分析

设算法的运行时间为 $T(n)$

- 在select过程的步骤1和步骤2耗费的时间都是 $\Theta(1)$
- 步骤3为 $\Theta(n)$ 时间。
- 由于对每组排序需要时间为常量，步骤4耗费 $\Theta(n)$ 时间，
- 步骤5所需的时间为 $T(\lfloor n/5 \rfloor)$ ，
- 步骤6需要 $\Theta(n)$ 时间。
- 由以上的分析，步骤7耗费的时间至多是 $T(0.7n+1.2)$ 。希望用底函数和丢掉常数1.2来表示这个比率。为此，假设 $0.7n + 1.2 \leq \lfloor 0.75n \rfloor$ ，这样，得到 $0.7n + 1.2 \leq 0.75n - 1$ 。
- 当 $n \geq 44$ 时， $0.7n + 1.2 \leq 0.75n - 1$ 这个不等式将成立。
- 得出的结论为，对于 $n \geq 44$ ，步骤7所耗费的时间至多是 $T(\lfloor 0.75n \rfloor)$ 。

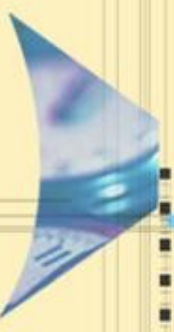
算法分析

上述分析蕴含着算法SELECT的运行时间有以下的递推式

$$T(n) \leq \begin{cases} c & \text{if } n < 44 \\ T(\lfloor n/5 \rfloor) + T(\lfloor 3n/4 \rfloor) + cn & \text{if } n \geq 44 \end{cases}$$

$$T(n) \leq \frac{cn}{1 - 1/5 - 3/4} = 20cn$$

定理3.4 在一个由n个元素组成的线序集合中提取第k小元素，所需的时间是 $\Theta(n)$ ，特别地，n个元素的中值可以在 $\Theta(n)$ 时间找出。



3.4.2 快速排序

- 快速排序算法，QUICKSORT，是一种非常流行且高效的排序算法，该算法具有 $\Theta(n \log n)$ 的平均运行时间。
- 快速排序算法优于MERGESORT的一点在于它在本位上排序，即不需要辅助的存储空间。
- 快速排序算法的基础是划分算法：SPLIT

划分算法

- 基本思想

- 设 $A[\text{low}..\text{high}]$ 是一个包含 n 个数的数组，并且 $x = A[\text{low}]$.
- 重新安排 A 中元素，使得小于等于 x 的元素都在 x 的前面，而大于 x 的元素都在 x 的后面。
- 在数组中的元素改变排列方式后，对于某个 w , $\text{low} \leq w \leq \text{high}$, x 在 $A[w]$ 中。

例如： $A = \{5, 3, 9, 2, 7, 1, 8\}$. 重新安排其中的元素后，
 $A = \{1, 3, 2, 5, 7, 9, 8\}$, $w = 4$.

- 重新排列的操作称为围绕 x 的拆分或划分， x 称为主元或者拆分元。



A partitioning algorithm

- **定义3.1** 如果元素 $A[j]$ 既不小于 $A[\text{low}..j-1]$ 中的元素, 也不大于 $A[j+1..\text{high}]$ 中的元素, 则称其处在一个适当的位置或正确位置。
- **观察结论 3.2** 用 x ($x \in A$)作为主元划分数组 A 后, x 将处在一个正确位置。



Algorithm 3.5 SPLIT

Input: An array of elements $A[\text{low}..\text{high}]$.

Output: (1) A with its elements rearranged, if necessary, as described above.

(2) w , the new position of the splitting element $A[\text{low}]$.

1. $i \leftarrow \text{low}$

2. $x \leftarrow A[\text{low}]$

3. **for** $j \leftarrow \text{low}+1$ **to** high

4. **if** $A[j] \leq x$ **then**

5. $i \leftarrow i+1$

6. **if** $i \neq j$ **then** interchange $A[i]$ and $A[j]$

7. **end if**

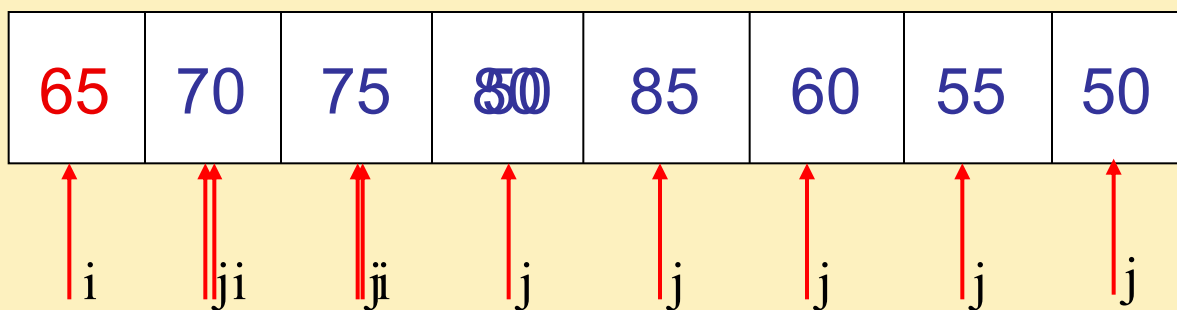
8. **end for**

9. interchange $A[\text{low}]$ and $A[i]$

10. $w \leftarrow i$

11. **return** A and w

举例



x=

65

Success!



SPLIT算法分析

- 观察结论 5.3

由算法SPLIT执行的元素比较次数恰好是 $n-1$. 因此, 它的时间复杂性为 $\Theta(n)$.

算法的空间复杂性为 $\Theta(1)$.



QUICKSORT排序算法

- 基本思想

- 通过主元 $A[\text{low}]$ 对 $A[\text{low}..\text{high}]$ 重排, 使得 $A[\text{low}]$ 处于它的正确位置。所有小于等于它的元素在 $A[\text{low}..\text{w}-1]$ 中, 所有大于它的元素在 $A[\text{w}+1..\text{high}]$ 中.
- 对两个子数组 $A[\text{low}..\text{w}-1]$ 和 $A[\text{w}+1..\text{high}]$ 递归排序, 最后自动产生整个排好序的数组。

Algorithm 3.6 QUIUCKSORT

Input: An array $A[1..n]$ of n elements.

output: The elements in A sorted in nondecreasing order.

1. *quicksort*($A, 1, n$)

Procedure *quicksort*($A, low, high$)

1. **if** $low < high$ **then**

2. *SPLIT*($A[low..high], w$) { w is the new position of $A[low]$ }

3. *quicksort*($A, low, w-1$)

4. *quicksort*($A, w+1, high$)

5. **end if**

举例

- 设数组 $A=\{4,6,3,1,8,7,2,5\}$.

4	6	3	1	8	7	2	5
2	3	1	4	8	7	6	5

2	3	1
1	2	3

8	7	6	5
5	7	6	8

1
1

3
3

5	7	6
5	7	6

7	6
6	7

6
6

Success!





MERGESORT 和 QUICKSORT

- 两个排序算法各自都包含一个基本算法的一系列调用，分别是： *MERGE* and *SPLIT*.
- 但从算法的观点看，在 MERGESORT 中，合并排序序列属于组合步骤，而在 QUICKSORT 中的划分过程属于划分步骤。事实上，QUICKSORT 中不存在组合步骤。

算法分析

- 空间复杂性

- 尽快不需要辅助数组，但QUICKSORT算法还是需要 $O(n)$ 的空间。这是因为在每次递归中，必须要保存数组右边部分的左右边界 $w+1$ 和 $high$ 。其所需工作空间在 $\Theta(\log n)$ 和 $\Theta(n)$ 之间变化。

- 时间复杂性

定理3.5 在最坏情况下，算法QUICKSORT的运行时间是 $\Theta(n^2)$ 。然而，如果总是选择中项为主元，则时间复杂性为 $\Theta(n \log n)$ 。

定理 3.6 QUICKSORT 算法对 n 个元素的数组排序执行的平均元素比较次数为 $\Theta(n \log n)$ 。

3.4.3 多数元素（众数）问题

- 问题描述：
- 给定一个大小为 n 的数组，找到其中的多数元素。多数元素（众数）是指在数组中出现次数大于 $\lfloor n/2 \rfloor$ 的元素。
- 假设数组是非空的，并且给定的数组总是存在多数元素。设计分治算法找出数组中的多数元素。
 - 首先将给定的数组分成两部分【分解】；
 - 分别求解这两部分的多数元素【求解】；
 - 比较这两部分的多数元素，最终得出结果【合并】

3.4.3 多数元素问题

基本思想:

- 根据多数元素的定义可知多数元素必定在数组中出现次数最多，而若将原数组分成2部分，则在其中一部分出现次数最多的必定是多数元素，证明如下：
 - 假定数组长度为 n ，多数元素为 a ，出现次数为 $f(a)$ ，已知： $f(a) > n/2$
 - 将数组分成两部分，长度分别为 l ， r ，其中 $l+r = n$
 - 若假设 $f_l(a) \leq l/2$ ， $f_r(a) \leq r/2$ 则必定 $f(a) \leq n/2$ ，与已知 a 为多数元素相矛盾，故假设不成立
 - 故将原数组分成2部分，则在其中一部分出现次数最多的必定是多数元素

算法3.7 MajorityElement

输入: n 个元素的数组 $A[1..n]$, 且 A 中必定存在多数元素

输出: 数组 A 中的多数元素

1. majorityElementRec($A, 1, n$)

过程 majorityElementRec ($A, low, high$)

1. **if** $low = high$ **then** return $A[low]$

2. $mid = (high + low) / 2$

3. $left = \text{majorityElementRec}(A, low, mid)$ //左半部分最多元素

4. $right = \text{majorityElementRec}(A, mid + 1, high)$ //右半部分最多元素

5. **if** $left = right$ **then** return $left$

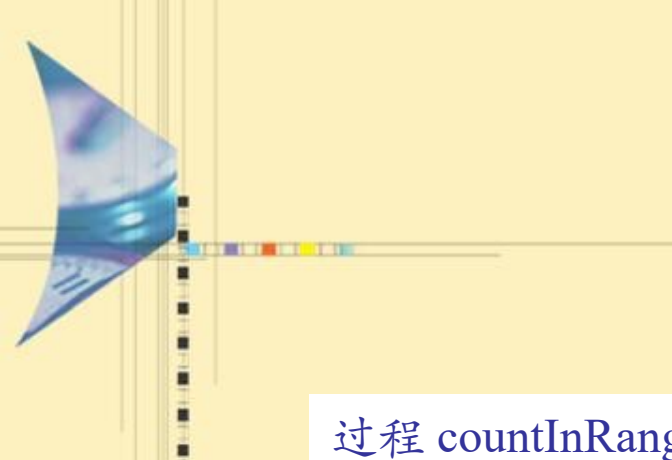
6. $leftCount = \text{countInRange}(A, left, low, high)$ //left在左边出现次数

7. $rightCount = \text{countInRange}(A, right, low, high)$ //right在右边出现次数

8. **if** $leftCount > rightCount$ **then** return $left$ //出现次数更多的为多数元素

9. **else** return $right$

10. **end if**



过程 countInRange($A, c, \text{low}, \text{high}$)

1. $\text{count} = 0;$
2. for $i = \text{low}$ to high
3. if $A[i] = c$ then
4. $\text{count} \leftarrow \text{count} + 1$
5. end if
6. end for
7. return $\text{count};$



算法分析

- 时间复杂度: $O(n \log n)$

$$C(n) = C(\lfloor n/2 \rfloor) + C(\lceil n/2 \rceil) + \Theta(n); \quad C(1) = 0$$

- 空间复杂度: $O(\log n)$

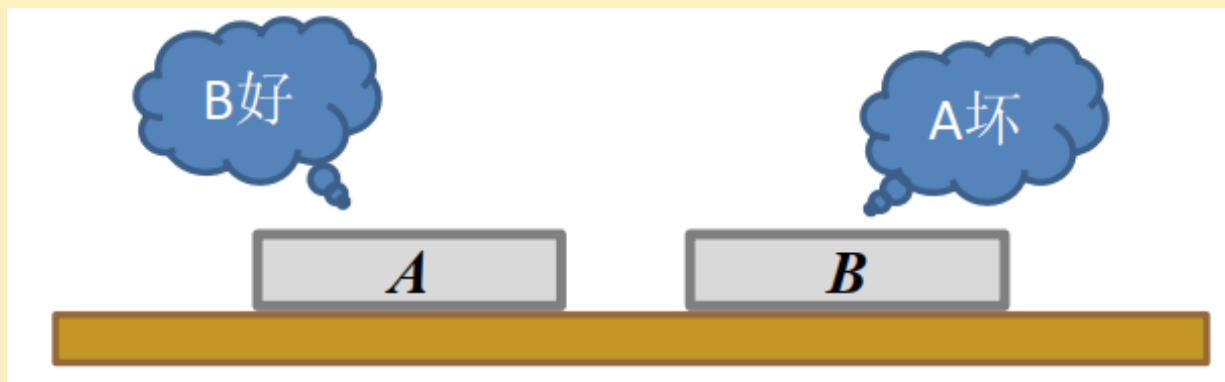


3.4.4 芯片测试问题

问题描述： 有 n 片芯片，已知其中好芯片至少比坏芯片多1片,需要通过测试从中找出1片好芯片。

测试方法：

将芯片放到测试台上，2片芯片互相测试，并报告结果“好”或“坏”。好芯片报告结果一定是正确的，坏芯片的报告不可靠（可能对也可能不对）。如何设计一个算法，使用最少测试次数，从中挑出1片好芯片？



A、B为芯片。测试方法为：将2片芯片（A和B）置于测试台上，互相进行测试，测试报告是“好”或者“坏”，只取其一。

测试结果四种可能

<i>A</i> 报告	<i>B</i> 报告	结论
<i>B</i> 是好的	<i>A</i> 是好的	<i>A, B</i> 都好或 <i>A, B</i> 都坏
<i>B</i> 是好的	<i>A</i> 是坏的	至少一片是坏的
<i>B</i> 是坏的	<i>A</i> 是好的	至少一片是坏的
<i>B</i> 是坏的	<i>A</i> 是坏的	至少一片是坏的

- 输入： n 片芯片，其中好芯片，至少比坏芯片多一片
- 问题：设计一种测试方法，通过测试从 n 片中挑出1片好芯片
- 要求：使用最少的测试次数

如何测试一块芯片的好坏？

问题：从 n 片芯片中，测试出 A 是好芯片还是坏芯片？

方法：用其他 $n-1$ 片芯片对 A 进行测试。

根据条件，好芯片至少比坏芯片多1片，有如下事实：

- 1.如果 n 是奇数：好芯片数 $\geq (n+1) / 2$
- 2.如果 n 是偶数：好芯片数 $\geq n/2 + 1$

例1: $n=7$ (奇数): 好芯片数 ≥ 4

用其他6个芯片测试A, 根据A是好芯片还是坏芯片, 有两种情形:

情形1: A是好芯片, 则6个芯片中至少3个报告“好”

情形2: A是坏芯片, 则6个芯片中至少4个报告“坏”

推广到一般, 对于 n 是奇数的情况:

好芯片数 $\geq (n+1)/2$

如果A是好芯片, 则至少有 $(n-1)/2$ 个报“好”;

如果A是坏芯片, 则至少有 $(n+1)/2$ 个报“坏”。

因此: $n-1$ 个报告中, 如果至少一半报“好”, 则A是好芯片;
如果超过一半报“坏”, 则A是坏芯片。



例2: $n=8$ (偶数): 好芯片数 ≥ 5

用其他7个芯片测试A, 根据A是好芯片还是坏芯片, 有两种情形:

情形1: A是好芯片, 则7个芯片中至少4个报告“好”

情形2: A是坏芯片, 则7个芯片中至少5个报告“坏”

推广到一般, 对于 n 是偶数的情况:

好芯片数 $\geq n/2 + 1$

如果A是好芯片, 则至少有 $n/2$ 个报“好”;

如果A是坏芯片, 则至少有 $n/2 + 1$ 个报“坏”。

因此: $n-1$ 个报告中, 如果超过一半报“好”, 则A是好芯片; 如果超过一半报“坏”, 则A是坏芯片。

蛮力算法

基本思想：

任取 1 片，用其他所有芯片对它进行测试，如果是好芯片，则测试结束；如果是坏芯片，则抛弃，再从剩下芯片中任取 1 片进行测试，直到得到 1 片好芯片为止。

算法时间估计：

首先选第一片测试：需测试 $n-1$ 次；

如果第一片是坏芯片，再需测试 $n-2$ 次；

如果第二片还是坏芯片，还需测试 $n-3$ 次

...

时间复杂度： $O(n^2)$



分治算法（假设 n 为偶数）

基本思想：

假设 n 为偶数，将 n 片芯片两两一组做测试淘汰，剩下芯片构成子问题，进入下一轮分组淘汰。

淘汰规则：

报告结果为“好，好” $\implies$ 任留1片，报告结果为其他情况 $\implies$ 全部抛弃。进入下一轮递归，截止条件： $n \leq 3$

如果 $n=3$ ，则一次测试可得到好芯片；

如果 $n=1$ 或 2 ，不需要再测试，他们都为好芯片，任意取1片即可。

分治策略正确性

分治策略的正确性的基本条件是：子问题与原问题性质相同。

命题：当 n 是偶数时，在上述淘汰规则下，经过一轮淘汰后，剩下的芯片中好芯片比坏芯片至少多1片。

证明：

假设原问题中分组后A，B都好的芯片有 i 组，A与B一好一坏的有 j 组，A与B都坏的有 k 组。初始芯片总数 $2i+2j+2k=n$ 。

经过一轮淘汰后，好芯片还剩 i 片(i 组中留1片)，坏芯片最多还剩 k 片 (j 中全部淘汰，因为每一对中至少一个报告“坏” (好芯片) 的，而 k 组中最多留 k 个，对应都错误报告为“好”的情形)。

又因为，初始好芯片数($2i+j$) 多于坏芯片($j+2k$)，即

$$2i+j > j+2k, \text{ 故: } i > k$$

因此，剩余的芯片中好芯片 i 比坏芯片 k 至少多1片。

命题成立。

如果n是奇数怎么办？

分治策略的正确性的基本条件是：子问题与原问题性质相同。
当n是奇数时，可能会出现子问题与原问题形式不同。

输入： 好 好 好 好 坏 坏 坏
分组： 好 好 好 好 坏 坏 坏
淘汰后： 好 好 坏 坏

淘汰后的子问题不满足于原问题性质相同，此时无法继续测试。
处理办法：当n为奇数时，增加一轮对轮空芯片的单独测试，如果该轮空芯片为好芯片则算法结束，如果是坏芯片，则淘汰该芯片。

算法3.8 CHIPTEST

输入：n片芯片组成的数组，其中好芯片比坏芯片至少多1片

输出：1片好芯片

```
1.  $k \leftarrow n$ 
2. while  $k > 3$  do
3.   将芯片分成  $\lfloor k/2 \rfloor$  组
4.   如果k为奇数，则单独测试剩下芯片，如果是好芯片，返回结果;否则，舍弃该芯片
5.   for  $i=1$  to  $\lfloor k/2 \rfloor$ 
6.     第i组的2个芯片互相测试
7.     if 2片“好”，then 任取1片留下
8.     else 2片同时丢掉
9.     end if
10.  end for
11.   $k \leftarrow$  剩下的芯片数
12. end while
13. if  $k = 3$ 
14. then 任取2片芯片测试
15.   if 至少1坏，then 取没测的芯片 //没测的肯定是好芯片
16.   else 任取1片被测芯片 //都是好芯片
17.   end if
18. end if
19. if  $k=2$  or 1 then 任取1片 end if //都是好芯片
```

算法分析

- 设输入规模为 n ，每轮分组测试时间 $O(n)$;
- 对于 n 为奇数的情况需要进一步处理, 处理时间为 $O(n)$; (不增加时间复杂度)
- 每轮淘汰后，芯片数至少减半。

设 $W(n)$ 表示最坏情况 n 片芯片测试的次数，则

$$W(n) = W(n/2) + O(n)$$

$$W(3)=1, W(2)=W(1)=0$$

因此， $W(n) = O(n)$

3.4.5 循环赛日程表

问题描述：

设有 $n = 2^k$ 个运动员要进行网球循环赛。现要设计一个满足以下要求的比赛日程表：

- (1) 每个选手必须与其他 $n-1$ 个选手各赛一次；
- (2) 每个选手一天只能赛一次；
- (3) 循环赛一共进行 $n-1$ 天。

3.4.5 循环赛日程表

基本思想：

- 假设 n 位选手被顺序编号为 $1, 2, \dots, n$ (2^k)
- 按问题要求可将比赛日程表设计成一个 n 行 $n-1$ 列的二维表，其中第 i 行、第 j 列表示和第 i 个选手在第 j 天比赛的选手。

$k=1$ 时, 只有2个选手

人为添加的
表示选手1与
选手2比赛

1	2
2	1

由 $k=1$ 创建 $k=2$ 的过程

$k=1$

1	2
2	1

加 $2^{k-1}=2$

左上角

右上角

1	2	3	4
2	1	4	3
3	4	1	2
4	3	2	1

$k=2$

左下角

右下角

由 $k=2$ 创建 $k=3$ 的过程

$k=2$

1	2	3	4
2	1	4	3
3	4	1	2
4	3	2	1

加 $2^{k-1}=4$

$k=3$

左上角

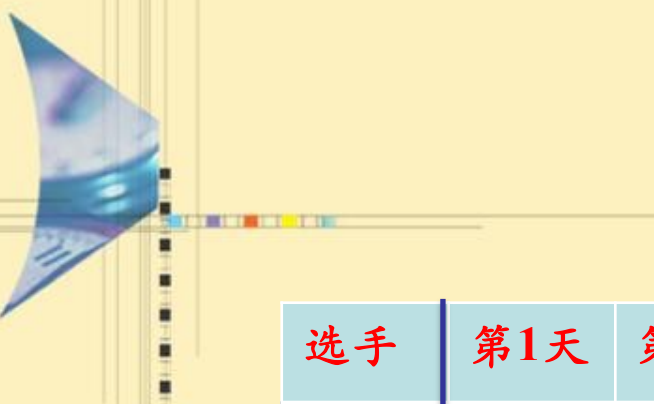
右上角

1	2	3	4	5	6	7	8
2	1	4	3	6	5	8	7
3	4	1	2	7	8	5	6
4	3	2	1	8	7	6	5
5	6	7	8	1	2	3	4
6	5	8	7	2	1	4	3
7	8	5	6	3	4	1	2
8	7	6	5	4	3	2	1

左下角

右下角





选手	第1天	第2天	第3天	第4天	第5天	第6天	第7天
1	2	3	4	5	6	7	8
2	1	4	3	6	5	8	7
3	4	1	2	7	8	5	6
4	3	2	1	8	7	6	5
5	6	7	8	1	2	3	4
6	5	8	7	2	1	4	3
7	8	5	6	3	4	1	2
8	7	6	5	4	3	2	1



将 $n=2^k$ 问题划分为4部分，均为 2^{k-1} ：

左上角：左上角为 2^{k-1} 个选手在前半程的比赛日程（ $k=1$ 时直接给出，否则，上一轮求出的就是 2^{k-1} 个选手的比赛日程）。

左下角：左下角为另外 2^{k-1} 个选手在前半程的比赛日程，由左上角加 2^{k-1} 得到，例如 2^2 个选手比赛，左下角由左上角直接加 $2(2^{k-1})$ 得到， 2^3 个选手比赛，左下角由左上角直接加 $4(2^{k-1})$ 得到。

右上角：将左下角直接复制到右上角得到另 2^{k-1} 个选手在后半程的比赛日程。

右下角：将左上角直接复制到右下角得到 2^{k-1} 个选手在后半程的比赛日程。

算法 3.9 PLAN

输入: $n=2^k$ 个选手组成的数组 $A[1..n]$; $k \geq 1$

输出: 比赛日程安排表 $P[n,n]$ 。第 i 行、第 j 列表示和第 i 个选手在第 j 天比赛的选手

```
1.   $n \leftarrow 2$ ;                                //  $n$  从  $2^1=2$  开始
2.   $P[1][1]=1$ ;  $P[1][2] \leftarrow 2$            // 求解 2 个选手比赛日程, 得到左上角元素
3.   $P[2][1] \leftarrow 2$ ;  $P[2][2] \leftarrow 1$ 

4.  for  $t \leftarrow 1$  to  $k$                         // 迭代处理  $2^2(t=1) \dots, 2^k(t=k-1)$  个选手
5.       $temp \leftarrow n$                                 //  $temp=2^t$ 
6.       $n \leftarrow n*2$                                 //  $n=2^{(t+1)}$ 
7.      for  $i \leftarrow temp+1$  to  $n$                 // 填左下角元素
8.          for  $j \leftarrow 1$  to  $temp$ 
9.               $P[i][j] \leftarrow P[i-temp][j]+temp$     // 产生左下角元素
10.         end for
11.     end for
12.     for  $i \leftarrow 1$  to  $temp$                 // 填右上角元素
13.         for  $j \leftarrow temp+1$  to  $n$ 
14.              $P[i][j] \leftarrow P[i+temp][(j+temp)\% n]$ 
15.         end for
16.     end for
17.     for  $i \leftarrow temp+1$  to  $n$                 // 填右下角元素
18.         for  $j \leftarrow temp+1$  to  $n$ 
19.              $P[i][j] \leftarrow P[i-temp][j-temp]$ 
20.         end for
21.     end for
22. end for
```

3.4.6 求向量卷积

- 向量计算:

给定向量 $a = (a_0, a_1, \dots, a_{n-1})$ 和 $b = (b_0, b_1, \dots, b_{n-1})$

向量和 $a + b = (a_0 + b_0, a_1 + b_1, \dots, a_{n-1} + b_{n-1})$

内积 $a \cdot b = a_0 b_0 + a_1 b_1 + \dots + a_{n-1} b_{n-1}$

卷积 $a * b = (c_0, c_1, \dots, c_{2n-2})$, 其中

$$c_k = \sum_{\substack{i+j=k \\ i,j < n}} a_i b_j, \quad k = 0, 1, \dots, 2n-2$$

由 a 与 b 的元素形成的下述矩阵“面”沿副对角线方向卷了起来得到的“一束”向量。

$a_0 b_0$	$a_0 b_1$		$a_0 b_{n-2}$	$a_0 b_{n-1}$
$a_1 b_0$	$a_1 b_1$		$a_1 b_{n-2}$	$a_1 b_{n-1}$
.....				
$a_{n-1} b_0$	$a_{n-1} b_1$		$a_{n-1} b_{n-2}$	$a_{n-1} b_{n-1}$

卷积等价于多项式相乘

多项式乘法:

给定多项式

$$A(x) = a_0 + a_1x + a_2x^2 + \dots + a_{m-1}x^{m-1}$$

$$B(x) = b_0 + b_1x + b_2x^2 + \dots + b_{n-1}x^{n-1}$$

计算 $C(x) = A(x) B(x)$

$$\begin{aligned} &= a_0b_0 + (a_0b_1 + a_1b_0)x + (a_0b_2 + a_1b_1 + a_2b_0)x^2 \\ &\quad + \dots + a_{m-1}b_{n-1}x^{m+n-2} \end{aligned}$$

其中 x^k 的系数

$$c_k = \sum_{\substack{i+j=k \\ i \in \{0,1,\dots,m-1\} \\ j \in \{0,1,\dots,n-1\}}} a_i b_j \quad k = 0, 1, \dots, m+n-2$$

卷积计算

给定向量 $a = (a_0, a_1, \dots, a_{n-1})$ 和 $b = (b_0, b_1, \dots, b_{n-1})$

直接计算: $O(n^2)$

快速算法: 卷积计算等价于多项式相乘.

$$A(x) = a_0 + a_1x + a_2x^2 + \dots + a_{m-1}x^{m-1}$$

$$B(x) = b_0 + b_1x + b_2x^2 + \dots + b_{n-1}x^{n-1}$$

$$C(x) = A(x)B(x) = a_0b_0 + (a_0b_1 + a_1b_0)x + (a_0b_2 + a_1b_1 + a_2b_0)x^2 \\ + (\dots + a_{m-1}b_{n-1})x^{m+n-2}, \quad C(x) \text{ 的系数向量就是 } a * b.$$

设计思想:

1. 选择特定值 $x_1, x_2, \dots, x_{2n}$, 求出 $A(x_j)$ 和 $B(x_j)$, $j=1, 2, \dots, 2n$
2. 对每个 j , 计算 $C(x_j)$
3. 利用多项式插值方法, 根据 $C(x)$ 在 $x=x_1, x_2, \dots, x_{2n}$ 的值求出多项式 $C(x)$

$2n$ 个数的选择：1 在复数域的 $2n$ 次方根

$$\omega_j = e^{\frac{2\pi j}{2n}i} = e^{\frac{\pi j}{n}i} = \cos \frac{\pi j}{n} + i \sin \frac{\pi j}{n} \quad j=0,1,\dots,2n-1$$

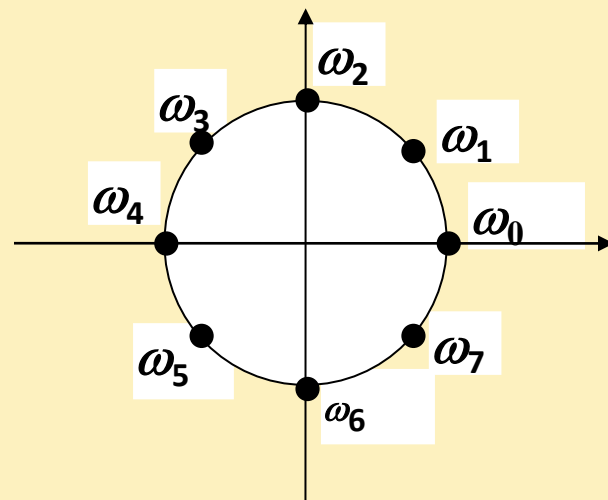
例如 $n=4$, 1的8次方根是:

$$\omega_0 = 1, \quad \omega_1 = e^{\frac{\pi i}{4}} = \frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}i,$$

$$\omega_2 = e^{\frac{2\pi i}{4}} = i, \quad \omega_3 = e^{\frac{3\pi i}{4}} = -\frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}i,$$

$$\omega_4 = e^{\pi i} = -1, \quad \omega_5 = e^{\frac{5\pi i}{4}} = -\frac{\sqrt{2}}{2} - \frac{\sqrt{2}}{2}i,$$

$$\omega_6 = e^{\frac{6\pi i}{4}} = -i, \quad \omega_7 = e^{\frac{7\pi i}{4}} = \frac{\sqrt{2}}{2} - \frac{\sqrt{2}}{2}i$$



多项式求值算法

给定多项式: $A(x) = a_0 + a_1x + a_2x^2 + \dots + a_{n-1}x^{n-1}$

设 x 为 1 的 $2n$ 次方根, 对所有的 x 计算 $A(x)$ 的值.

算法1: 对每个 x 做下述运算:

依次计算每个项 $a_i x^i$, 对 i 求和得到 $A(x)$,
 $T_1(n) = O(n^3)$ (对一个特定 x 是 $O(n^2)$, 共 $2n$ 个)

算法2: $A_1(x) = a_{n-1}$

$$A_2(x) = a_{n-2} + xA_1(x)$$

$$A_3(x) = a_{n-3} + xA_2(x)$$

...

$$A_n(x) = a_0 + xA_{n-1}(x) = A(x) \quad (\text{Horner 规则})$$

$$T_2(n) = O(n^2)$$



多项式求值算法-分治算法

基本思想:

$$A_{\text{even}}(x) = a_0 + a_2x + a_4x^2 + \dots + a_{n-2}x^{(n-2)/2} \quad (\text{偶数下标系数组成}) \quad (1)$$

$$A_{\text{odd}}(x) = a_1 + a_3x + a_5x^2 + \dots + a_{n-1}x^{(n-2)/2} \quad (\text{奇数下标系数组成}) \quad (2)$$

$$A(x) = A_{\text{even}}(x^2) + x A_{\text{odd}}(x^2), \quad x^2 \text{ 为 } 1 \text{ 的 } n \text{ 次根} \quad (3)$$

算法3:

1. 计算1 的所有的 $2n$ 次根
2. 分别计算 $A_{\text{even}}(x^2)$ 与 $A_{\text{odd}}(x^2)$
3. 利用步2 的结果计算 $A(x)$

复杂度分析: $T(n) = T_1(n) + f(n)$, $f(n) = O(n)$ 计算 $2n$ 次根时间

$$T_1(n) = 2T_1(n/2) + g(n), \quad g(n) = O(n), \text{ 对所有根 } x, \text{ 通过(3)计算 } A(x) \text{ 时间}$$

$$T_1(1) = O(1)$$

$$T(n) = O(n \log n)$$

卷积计算的时间分析

步1: 求值 $A(\omega_j)$ 和 $B(\omega_j)$, $j=0,1,\dots,2n-1$. $O(n\log n)$

步2: 计算 $C(\omega_j)$, $j=0,1,\dots,2n-1$. $O(n)$

步3: 构造多项式

$$D(x)=C(\omega_0)+C(\omega_1)x+\dots+C(\omega_{2n-1})x^{2n-1}$$

可以证明: $D(\omega_0)=2nc_0 \implies c_0 = D(\omega_0)/2n$

$$D(\omega_j)=2nc_{2n-j}, \quad j=1, \dots, 2n-1 \implies c_{2n-j} = D(\omega_j)/2n$$

即只要计算出所有的 $D(\omega_j)$, 就得到所有的 c_{2n-j} , $j=0,1,\dots,2n-1$.
 $O(n\log n)$

总的时间为 $O(n\log n)$

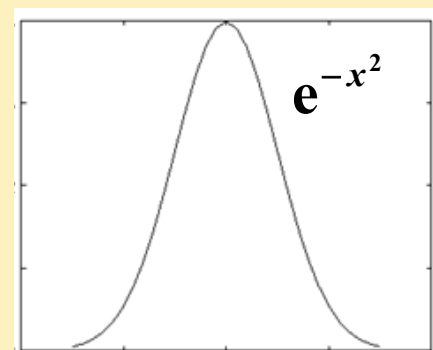
卷积应用：信号处理

给定序列 $a=(a_0,a_1,\dots,a_{m-1})$ ，表示信号在时刻 $0,1,\dots,m-1$ 的度量值，由于噪音干扰，需要平滑处理，即对值 a_i ，用其前后 k 步内的值进行加权平均。离 a_i 越近权值越大，处理结果为 a_i' 。

高斯滤波的权值函数为

$$w_s = \frac{1}{z} e^{-s^2}, \quad s = 0, \pm 1, \dots, \pm k$$

$$w = (w_{-k}, \dots, w_{-1}, w_0, w_1, \dots, w_k)$$



其中 z 用于归一化处理， k 使得所有的权值之和为1。

处理结果为
$$a_i' = \sum_{s=-k}^k a_{i+s} w_s$$

例

$a=(a_0,a_1,\dots,a_8)$, $k=2$, $w=(w_{-2},w_{-1},w_0,w_1,w_2)=(b_4,b_3,b_2,b_1,b_0)$

$a_i'=a_{i-2}b_4+a_{i-1}b_3+a_ib_2+a_{i+1}b_1+a_{i+2}b_0$, 下标之和为 $i+k$

a_0b_0	a_0b_1	a_0b_2	a_0b_3	a_0b_4	a_2'
a_1b_0	a_1b_1	a_1b_2	a_1b_3	a_1b_4	a_3'
a_2b_0	a_2b_1	a_2b_2	a_2b_3	a_2b_4	a_4'
a_3b_0	a_3b_1	a_3b_2	a_3b_3	a_3b_4	a_5'
a_4b_0	a_4b_1	a_4b_2	a_4b_3	a_4b_4	a_6'
a_5b_0	a_5b_1	a_5b_2	a_5b_3	a_5b_4	
a_6b_0	a_6b_1	a_6b_2	a_6b_3	a_6b_4	
a_7b_0	a_7b_1	a_7b_2	a_7b_3	a_7b_4	
a_8b_0	a_8b_1	a_8b_2	a_8b_3	a_8b_4	

平滑处理

定义向量

$$a=(a_0, a_1, \dots, a_{m-1})$$

$$b=(b_{2k}, b_{2k-1}, \dots, b_0)$$

$$=(w_{-k}, w_{-k+1}, \dots, w_k)=w$$

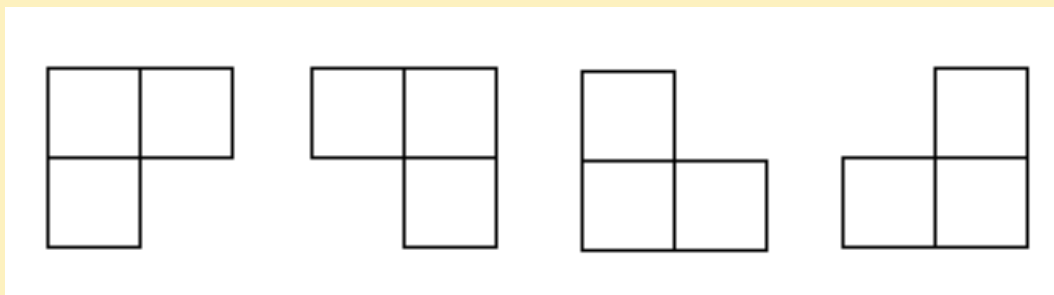
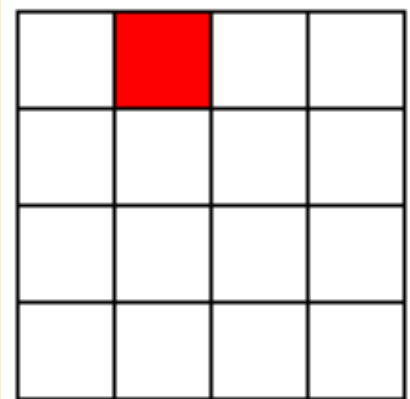
$$a_i' = \sum_{s=-k}^k a_{i+s} b_{k-s} = \sum_{s=-k}^k a_{i+s} w_s$$

把权向量 $b=w$ 看作 $2k+1$ 长度的窗口
在 a 上移动计算卷积.



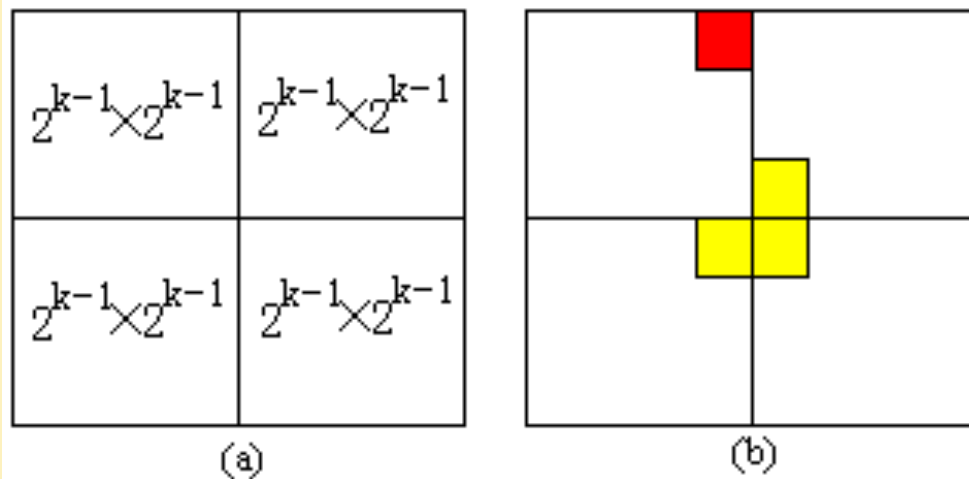
3.4.7 棋盘覆盖

在一个 $2^k \times 2^k$ 个方格组成的棋盘中，恰有一个方格与其它方格不同，称该方格为一特殊方格，且称该棋盘为一特殊棋盘。在棋盘覆盖问题中，要用图示的4种不同形态的L型骨牌覆盖给定的特殊棋盘上除特殊方格以外的所有方格，且任何2个L型骨牌不得重叠覆盖。



3.4.7 棋盘覆盖

- 当 $k > 0$ 时，将 $2^k \times 2^k$ 棋盘分割为4个 $2^{k-1} \times 2^{k-1}$ 子棋盘。特殊方格必位于4个较小的子棋盘之一中，其余3个子棋盘中无特殊方格。
- 为了将这3个无特殊方格的子棋盘转化为特殊棋盘，可以用一个L型骨牌覆盖这3个较小棋盘的会合处，如(b)所示，从而将原问题转化为4个较小规模的棋盘覆盖问题。递归地使用这种分割，直至棋盘简化为棋盘 1×1 。



3.4.8 最大子序列和问题

问题描述：输入一组整数，求出这组数字的**子序列和**中的最大值。也就是求出最大子序列的和。

例如：

输入序列：-2， 11， -4， 13， -5， -2， 则最大子序列和为20。



3.4.8 最大子序列和问题

基本思路

- 将数组分为大小平衡的两部分，
- 分别求左半部分最大子序列和，和右半部分的最大子序列和
- 最大子序列可能出现在左半部分，右半部分，或者是两者之间。（取三者中最大的）
- 为求两者之间的最大子序列和，先对左半部分求以最后一个数字为结尾的最大序列和。然后对右半部分以第一个数字开始算最大序列和，将两者加起来即是。